

Videos Feed — A Self-Adapting Ranking Algorithm

A shoppable-video feed that learns each user's patterns on its own and keeps improving as the user base grows — without being rebuilt at every stage of growth.

Status: Approved, not yet implemented Date: 13 August 2026 Module: `src/modules/videos`

Context

The videos feed today is strictly chronological: `GET /api/videos` orders by `(created_at DESC, id DESC)` over `isActive = true`, keyset-paginated. ADR-031 chose this deliberately and deferred likes/comments over moderation cost, noting *"a manual feed-curation override is easy to add later; nothing here forecloses it."*

The goal: a feed algorithm that learns users' patterns on its own and keeps getting better as the user base grows — not something rebuilt at each stage of growth.

Why not Instagram Reels' architecture

Reels ranks on watch time because Instagram sells ads against attention. Adorini sells clothes: a viewer who watches twenty clips and buys nothing is a *success* for Instagram and a *failure* for us.

Meta's funnel (two-tower retrieval → distilled first-stage ranker → MTML second-stage → rerank) narrows *billions* of candidates with models retrained hourly on billions of interactions, served from FAISS/HNSW. We have **5 seeded products, 0 users, 0 orders, 0 videos, 0 engagement events**. A two-tower model trained on nothing ranks worse than sorting by date.

Worth borrowing is the *shape*, not the models: the funnel, the **Value Model** ($\sum W_i \times P(\text{event}_i)$), the diversity rerank, and the "audition" that guarantees new content early impressions.

What "self-adapting" actually requires

The instinct to bolt ML on later is the trap: it means a rewrite at every growth stage, and each rewrite needs data that was never logged. The property wanted here — *automatically* getting more personalised as data accumulates, with no manual switch — is not an ML feature. It is a property of **hierarchical (partial-pooling) Bayesian estimation**, and it comes for free:

When we know nothing about a user, their estimate is their segment's estimate. When we know nothing about the segment, it is the global estimate. As evidence accumulates at any level, that level's estimate automatically takes over — continuously, with no thresholds and no phase transitions.

That is the whole mechanism. It is one formula, not three systems.

The algorithm: hierarchical contextual Thompson sampling

Not a downgrade from ML — for a catalogue of 10–10,000 items with sparse data this is the *stronger* choice: it solves cold-start by construction (a no-data video has a wide posterior, so it gets sampled high often enough to

earn its audition), self-corrects without retraining, and provably balances explore/exploit with logarithmic regret.

Three nested posteriors, each the prior for the one below

For video v , user u , segment s , with S = value-weighted successes and F = failures:

```
global     $\alpha_g = 1 + S_v$                  $\beta_g = 1 + F_v$ 
segment    $\alpha_s = k_s \cdot \mu_g + S_{(v,s)}$      $\beta_s = k_s \cdot (1 - \mu_g) + F_{(v,s)}$ 
personal   $\alpha_p = k_p \cdot \mu_s + S_{(v,u)}$      $\beta_p = k_p \cdot (1 - \mu_s) + F_{(v,u)}$ 

 $\theta \sim \text{Beta}(\alpha_p, \beta_p)$            // Thompson draw - exploration is automatic

score =  $\theta \times \text{recencyDecay} \times \text{freshnessBoost} \times \text{stockHealth} \times \text{inventoryBalance}$ 
```

The trailing four multipliers are the business layer, specified below. Keeping them as *multipliers on a learned score* rather than as hard overrides is deliberate: the algorithm still decides what is good, the rules only shade it.

When a user has no history, $S_{(v,u)} = F_{(v,u)} = 0$ and the personal posterior collapses *exactly* to the segment posterior. Same for an unseen segment collapsing to global. k_s / k_p are pooling strengths — how much evidence it takes to outweigh the level above.

WHY THIS IS MAINTAINABLE BY A SMALL TEAM

The "model" is a table of counts. There are no model artifacts, no training pipeline, no feature store, no ANN index. Learning is incremental — the existing `@nestjs/schedule` rollup pattern (`cod-verification-sweep.service.ts`) just updates integers.

Segments

Start coarse, from data that exists on day one: gender, price band of past orders, top category, acquisition channel (referral vs organic). An unknown user falls into a segment that is the global posterior, so anonymous and brand-new users are handled by the same code path.

The reward, and why it also self-calibrates

Ultimate reward is **attributed revenue** — real money, but sparse. Proxy signals are dense but approximate. So the same partial-pooling trick applies to the weights themselves: start hand-set, and let a periodic regression of proxies against realised revenue pull them toward learned values, weighted by confidence. The weights below are the *starting prior*, not a permanent constant:

Event	Start weight	Rationale
Attributed purchase	100	The objective
Add to cart	25	Nearest leading indicator
Save	12	Private, un-gameable, genuine intent
Product tap	8	The tap is the point of a shoppable video
Share	6	Reels data: a send is $\approx 3\text{--}5\times$ a like for reach
Like	1	Cheapest to give, easiest to bot — deliberately near-floor
Watch-through $\geq 80\%$	3	Quality proxy only, never the goal

All live in env (`FEED_W_*`), tunable without a deploy.

Business rules

Rule 1 — New videos get pushed to the top

freshnessBoost : starts at `FEED_NEW_BOOST_MAX` (default 4.0) and decays to 1.0 as the video's audition is spent. The audition is spent on **whichever exhausts first — elapsed time or impressions delivered**:

```
progress      = max(ageHours / FEED_NEW_BOOST_HOURS, impressions / FEED_NEW_BOOST_IMPRESSIONS)
freshnessBoost = 1 + (FEED_NEW_BOOST_MAX - 1) × (1 - clamp(progress, 0, 1))
```

Both terms are load-bearing. **Time alone** means a video uploaded at 3am burns its whole boost overnight with nobody watching. **Impressions alone** mean a dud keeps its boost forever. Taking the max exhausts the boost on real exposure while capping how long it can hold prime position.

Additionally, **slot 1 of page 1 is reserved** for the newest un-audited video, so a new upload is genuinely the first thing a scrolling user sees rather than merely probable.

Three things this needs to not backfire

- **A concurrency cap.** If ten videos upload at once and all are boosted, the boost means nothing — they just re-sort chronologically among themselves. `FEED_NEW_BOOST_MAX_CONCURRENT` (default 3) limits how many hold boosted state; the rest queue and begin their audition as slots free up. Uploading a batch therefore staggers rather than floods.
- **Evidence still wins.** A boosted video with 500 impressions and no taps drops, because the boost exhausts on impressions while its posterior simultaneously sharpens downward. The boost buys a hearing, not a verdict.
- **A per-user repetition cap** (`FEED_MAX_REPEATS_PER_USER`, default 3). Without this, Rule 1 makes the feed *worse* for your most frequent users — they open the app five times a day and see the same new clip pinned at slot 1 every time. Needed regardless, but Rule 1 makes it urgent.

Rule 2 — Balance sales across inventory

Two distinct multipliers, because "don't push what's selling out" and "do push what isn't moving" are different problems with different risk profiles.

stockHealth — computed per video as the **max** over its tagged products (a video is still useful if *any* tagged product is buyable; using min would kill a good video over one sold-out accessory):

Condition	Multiplier
No tagged product buyable (inactive / zero stock)	0 — dropped entirely
<code>hoursToSellout < FEED_SELLOUT_URGENT_HOURS</code> (default 1)	0.15
<code>hoursToSellout < FEED_SELLOUT_LOW_HOURS</code> (default 6)	0.5
Healthy	1.0

where `hoursToSellout = currentStock / unitsSoldPerHour`. This is the direct answer to "sold in bulk, gone within a minute": velocity, not just stock level, is what triggers the demotion.

inventoryBalance — the flip side, gently promoting over-stocked, slow-moving products so exposure spreads across the catalogue. **Clamped to [0.8, 1.3]** and that clamp is the important part: pushing slow movers is in direct tension with maximising revenue, and unbounded it would teach the feed to showcase exactly the products nobody wants. It shades the ranking; it never overrides it.

Freshness and staleness are measured on different clocks, so **freshnessBoost** and **recencyDecay** do not double-count: the boost is impression-gated (spent on exposure), the decay is purely temporal (long-term aging, half-life in days).

Two-tier data, because "within a minute" is faster than any rollup job

- **Live, per request** — the feed already joins tagged products for the chips, so it reads real **stockQuantity** in that same query. The hard "is this buyable at all" check is therefore never stale. Correctness does not depend on a job having run.
- **Rolled up, ~5 min** — sales velocity (**unitsSoldPerHour** , **hoursToSellout**) from a new **product-velocity.entity.ts** , refreshed by the same scheduled job. Being a few minutes stale here only makes the *optimisation* slightly late, never the correctness.

This also needs an index on **order_items.product_id** , which does not exist today — a velocity aggregate is currently a full scan.

Guardrails — the part that makes auto-learning safe

A system that optimises itself will find shortcuts. These are not optional:

1. **Propensity logging.** Record the probability with which each impression was shown. Without it the algorithm only ever learns about what it chose to show, and the bias is uncorrectable after the fact. Free to log now, impossible to reconstruct later — the same argument as logging events at all.
2. **Reward is revenue, not conversion count.** Optimising conversion *rate* teaches the feed to show only the cheapest items. Paise is the unit that cannot be gamed this way.
3. **Guaranteed exploration floor.** Every video gets **FEED_MIN_AUDITION_IMPRESSIONS** before its posterior is trusted, and a fixed share of every page is reserved for new arrivals — otherwise a new product can never break in. This is exactly what Reels' rerank stage does.
4. **Position-bias correction.** Log **feedPosition** ; slot 1 always out-earns slot 9 regardless of quality.
5. **Explainability.** An admin-only **GET /admin/videos/:id/ranking** returning the posterior, counts and score components. A small team cannot debug a ranker it cannot inspect.
6. **Rate-based, never raw counts.** Raw counts are rich-get-richer; the **Beta** prior stops a 1-tap/2-impression video rocketing to the top.
7. **Measure what the business rules cost.** Both new rules deliberately trade revenue for something else — Rule 1 spends prime impressions on unproven videos, Rule 2 suppresses your best sellers on purpose. That is the right call, but only if it stays bounded. The rollup should record revenue-per-session split by boosted vs unboosted impressions, so the boost multipliers can be tuned against evidence instead of taste.

Decisions taken

1. **Revenue per session is primary**, watch-through folded in as a quality signal only.

2. **Likes and saves ship.** This does not overturn ADR-031 — that ADR's cost was *moderating free text*. Likes and saves carry no text, so there is nothing to moderate. Amend it, don't reverse it.
3. **Launch admin-curated (10–100 videos), architect for UGC.** Buyer uploads stay gated: the blockers are moderation, India's IT Rules 2021 (grievance officer, takedown windows) and transcoding — none of them code. Adding `uploaderId` + `moderationStatus` now makes enabling UGC a config change rather than a migration against a live table.

Phase A — Instrumentation (feed unchanged)

NON-NEGOTIABLE AND FIRST

The adaptive scorer cannot exist without this data, and every unlogged day is training data lost permanently.

New entities

video-event.entity.ts — append-only log:

Column	Notes
videoId	FK → videos CASCADE
userId	FK → users SET NULL, nullable — the feed is @Public(), anonymous viewers must log
sessionId	varchar(64) NOT NULL — client-generated UUID, gives anonymous continuity
eventType	enum: IMPRESSION, VIEW_START, WATCH_PROGRESS, COMPLETE, PRODUCT_TAP, LIKE, SAVE, SHARE, ADD_TO_CART, PURCHASE
watchMs	integer, nullable
feedPosition	smallint, nullable — position-bias correction
propensity	numeric, nullable — P(shown), for debiasing
orderId	FK → orders SET NULL, nullable — attributed purchase

Indexes (videoId, createdAt), (sessionId, videoId), plus a partial unique on (sessionId, videoId) WHERE event_type = 'IMPRESSION' so ingest is idempotent under client retries — the same two-layer pattern as uq_processed_webhook_*.

video-stat.entity.ts — rollup, PK videoId, one row per video per segment (segmentKey, with '__global__' as the global row — this is what makes the hierarchy cheap to read). Counters for impressions, views, completions, totalWatchMs, productTaps, likes, saves, shares, addToCarts, attributedOrders, attributedRevenuePaise, plus weightedSuccesses, updatedAt. Reads never aggregate the event log; that table only grows.

video-reaction.entity.ts — userId, videoId, kind enum LIKE | SAVE, @Unique('uq_video_reaction', ['userId', 'videoId', 'kind']) so toggling is idempotent.

Video gains (inert at launch): uploaderId FK → users SET NULL nullable (NULL = brand), and moderationStatus enum APPROVED | PENDING | REJECTED default APPROVED.

New routes

Method	Path	Auth
POST	/videos/events	@Public() + @OptionalUser() , batched, throttled hard
POST	/videos/:id/like	bearer, toggle
POST	/videos/:id/save	bearer, toggle
GET	/users/me/saved-videos	bearer — the save list doubles as a wishlist

VideosController is @Public() with no auth wiring today; the reaction routes are its first authenticated writes, so @CurrentUser / @OptionalUser need importing there.

Rollup: a @nestjs/schedule job recomputing video_stats every ~5 min from events newer than updatedAt .

Two bugs to fix while here, both found during exploration and both one-liners:

createVideoSchema.productIds lacks the .max(10) cap replaceVideoTagsSchema has; and video_product_tags.product_id has no index, so "videos for this product" on the PDP is a seq scan.

Phase B — The adaptive scorer

Implements the formula above. Once shipped it does not need replacing as the app grows — that is the point of choosing it.

Pagination is the genuinely hard part. The existing SeekCursor is {sortValue, id} with no sort discriminator, and keyset pagination needs a monotonic indexed column — a computed *and stochastic* score has neither. Solution: **session-seeded ordering**. On a cursorless request, score the catalogue, persist the ordered ID list in Redis under a feed-session key (TTL ~30 min), and let the cursor carry {sessionId, offset} . Pages stay stable while ranking stays live. Extend cursor.util.ts with a discriminated variant rather than overloading sortValue .

Redis fails fast (enableOfflineQueue: false) and RedisService exposes no counter or sorted-set primitives — OtpService already documents dropping to getClient() for this. Extend RedisService , and **degrade to the chronological feed on RedisProviderError** . The feed must never 503 because Redis blinked.

Rerank pass: no two consecutive videos tagging the same product; cap a product to ~2 appearances per page; drop videos whose every tagged product is inactive or out of stock — never show a clip that cannot be bought.

Explicitly not planned: two-tower embeddings, ANN indexes, learned deep rankers. Revisit only past ~10k videos with sustained traffic; the hierarchy above is what carries us there.

Files

Path	Change
database/entities/video-event.entity.ts , video-stat.entity.ts , video-reaction.entity.ts	New
database/entities/product-velocity.entity.ts	New — rollup: stock, unitsSoldPerHour , hoursToSellout (Rule 2)
database/entities/order-item.entity.ts	Add index on productId — velocity aggregate is a full scan today
database/entities/video.entity.ts	uploaderId , moderationStatus
database/entities/index.ts	Register the new entities
database/migrations/<ts>-AddVideoEngagement.ts	Generated, then run
modules/videos/services/feed-ranking.service.ts	New — hierarchical posteriors + Thompson draw. Pure, no I/O, fully unit-testable
modules/videos/services/video-events.service.ts	New — ingest + attribution
modules/videos/services/videos.service.ts	listFeed gains ranked path + chronological fallback
modules/videos/controllers/videos.controller.ts	Event, reaction, admin-explain routes
modules/jobs/services/video-stats-rollup.service.ts	New scheduled rollup + weight recalibration
common/pagination/cursor.util.ts	Discriminated session-cursor variant
providers/redis/redis.service.ts	List/counter primitives
config/env.validation.ts	FEED_W_* , FEED_RECENCY_HALFLIFE_DAYS , FEED_SESSION_TTL_SECONDS , FEED_MIN_AUDITION_IMPRESSIONS , FEED_POOLING_K_SEGMENT , FEED_POOLING_K_USER , plus Rule 1 (FEED_NEW_BOOST_MAX , _HOURS , _IMPRESSIONS , _MAX_CONCURRENT , FEED_MAX_REPEATS_PER_USER) and Rule 2 (FEED_SELLOUT_URGENT_HOURS , FEED_SELLOUT_LOW_HOURS , FEED_INVENTORY_BALANCE_MIN / _MAX) — all defaulted so no existing .env breaks
.gsd/DECISIONS.md	Amend ADR-031; new ADRs for hierarchical-Bayes-over-ML and session-seeded pagination

Two decisions needed before Phase A ships

1. **Anonymous tracking and the DPDP Act.** Logging watch behaviour for logged-out users is behavioural tracking of session-identifiable individuals. India's DPDP Act 2023 expects notice and a lawful basis. Needs a

privacy-notice line in the app and a retention policy on `video_events` — proposal: raw events purged at 90 days, rollups kept indefinitely since rollups are not personal data. Flag to `@ETHICS` .

2. **Attribution window.** How long after watching does a purchase still credit that video? Proposal: 7 days, session-scoped, last-touch. It directly drives the 100-weight term.

Verification

```
npm run migration:generate -- src/database/migrations/AddVideoEngagement
npm run migration:run
npm run migration:generate -- src/database/migrations/DriftCheck    # expect "No changes"
npx tsc --noEmit && npm run lint
npm test && npm run test:integration
npm run build && node dist/main.js
```

Unit — `feed-ranking.service.spec.ts` is the core, and must prove the self-adapting property rather than assume it: a user with zero history scores *identically* to their segment; a segment with zero history scores identically to global; as personal evidence is added the personal term provably takes over; a purchase outranks 100 likes; a zero-data video still surfaces (cold start); recency halves at the configured half-life; the `Beta` prior keeps a 1-tap/2-impression video off the top; diversity rerank never emits the same product twice consecutively. Ranking takes an **injected seeded RNG** so the stochastic path is deterministic under test.

Rule 1 specs — a freshly uploaded video takes slot 1; the boost decays to exactly 1.0 once *either* `FEED_NEW_BOOST_HOURS` elapses or `FEED_NEW_BOOST_IMPRESSIONS` is delivered, whichever comes first; a boosted video with many impressions and no engagement falls below an unboosted proven one (evidence beats boost); uploading ten at once boosts only `FEED_NEW_BOOST_MAX_CONCURRENT` ; the same user never sees one video more than `FEED_MAX_REPEATS_PER_USER` times.

Rule 2 specs — a product selling out inside the urgent window is demoted but *not* removed; a video is dropped only when **every** tagged product is unbuyable, and survives when one of several is sold out; demotion **reverses automatically once stock is replenished** (the multiplier is computed, never persisted); `inventoryBalance` never leaves `[0.8, 1.3]` however extreme the stock imbalance — a property test is better than an example here.

Integration — ingest is idempotent (same impression twice → one row); like/save round-trips; the feed degrades to chronological with Redis stopped rather than erroring; session pagination returns each video exactly once with no gaps; an out-of-stock-only video is dropped. Crucially, **selling the last unit removes the video from the very next request** without waiting for the rollup — that is the live-stock tier, and it is the one part of Rule 2 that must never be eventually consistent.

Live — there is *no* behavioural seed data today, so a synthetic generator must be written: seed ~30 videos and simulate a few thousand sessions with known preferences, then assert the feed converges toward the planted preference. That simulation is the only honest way to test a learning system before real users exist.